

Ambient Reachability Attacks

Trajectory-Level Vulnerabilities in Reasoning-Capable LLM Systems

DRAFT

John P. Alioto

December 2025

Abstract

We introduce a class of security vulnerability in large language model (LLM) systems that we term an *Ambient Reachability Attack* (ARA). ARAs do not require inputs whose content or form is designed to bypass or exploit model safety or alignment behavior. Every input in an ARA is benign. Every output is admissible under the model builder’s security regime. The model functions exactly as designed at all times.

The vulnerability ARAs identify lies in the trust architecture between a reasoning-capable model and an agent—human or automated—who acts upon its outputs. A threat actor deliberately places information in ambient content: documents, images, retrieved artifacts. In this way, the attack resembles a prompt injection attack with one critical difference: the injected content is completely benign in nature.

The mechanism of an ARA is that a correctly-functioning model discovers this information. The model may do this autonomously, without any user request, simply because its training disposes it to investigate structure. In agentic deployments, the model may act on what it discovers before any human is aware the interaction has begun. The attack uses helpful guidance at every turn. Yet the net result of the interaction trajectory is harm to the user as defined by their particular governance regime.

We argue this class of vulnerability is not reliably addressed by output filtering, refusal policies, or semantic analysis under the threat model we define. There is nothing to filter, refuse, or flag: each input is piecewise, locally helpful. The vulnerability exists only after the full trajectory where the user has taken an irreversible action that causes harm as evaluated post hoc against governance regimes that live outside the system.

We adopt the control-theoretic framework of reachability analysis to formalize this phenomenon. Each input irreversibly conditions the model’s state, committing to a trajectory prefix that constrains all future computation.

Finally, we distinguish *steering* (the mechanism) from *attack* (the evaluation). Steering is value-neutral: it is simply what inputs do (Vogel, 2025). Whether steering constitutes an attack depends on external governance regimes that are contested, jurisdiction-dependent, and often collapse to personal governance.

Contributions

This paper makes the following contributions:

- **Definition.** We define *Ambient Reachability Attack* (ARA) as trajectory-level harm under external governance with benign inputs and admissible outputs.
- **Diagnosis.** We identify the *compositionality assumption*—that pointwise safety implies trajectory safety—as the category error underlying current defenses.

- **Framework.** We map the phenomenon to control-theoretic vocabulary (reachable sets, disturbances, asymmetric information games) providing precise language for reasoning about this vulnerability class.
- **Implication.** We argue that defense requires *architectural independence*—an evaluator that cannot be corrupted by the content it evaluates. Root trust is one well-understood instantiation of this requirement.

1 Prompt Injection 2.0

ARA is an evolution of prompt injection. ARA takes the mechanisms of prompt injection to target a new vulnerability: the human-LLM interaction loop. This loop has vulnerabilities on both the human-side and the model-side. ARA seeks to hide human deficiencies from the model and model deficiencies from the human. The deception takes the form of helpfulness.

Note on Examples. We do not provide links or examples of attacks in the wild. It is not our goal to teach these approaches. This is a deliberate choice to weaken the paper in favor of not assuming all villains already wear black hats. Consider the damage done by script kiddies with published exploits. The industry is not mature enough to absorb vulnerabilities the way software engineering can. This attack cannot currently be stopped on the model provider-side, and thus we do not present working examples here.

2 Why Jailbreaks Work

To understand how ARA works, one must understand why jailbreaks work. The *how* of jailbreaks is not relevant. The *why* is critical.

When a token enters the model’s context, regardless of source, it perturbs the geometry of what is commonly referred to as the “manifold.” We prefer *reachability* language because it cleanly states the security property we care about.

Definition 2.1 (Reachable Set). The reachable set from state h under input class $\mathcal{X}_{\text{allowed}}$ is:

$$R(h) = \{h' \in \mathbb{R}^d : \exists(x_1, \dots, x_k) \in \mathcal{X}_{\text{allowed}}^k \text{ such that } h' = f_{\theta}^{(k)}(h, x_1, \dots, x_k)\}$$

This is the set of all states attainable from h via finite sequences of allowed inputs.

This is how LLMs are designed to work. This is by design.

Input tokens do not change $R(h_t)$. Input tokens change $R(h_{t+1})$. There is nothing special about the input tokens in ARA. They are indistinguishable from normal human-model interactions.

Jailbreaks, however, inject tokens that *are* distinguishable from normal interaction. The geometric distinction is precise. In prompt injection and jailbreaking, the attacker crafts perturbations of $R(h_{t+1})$ that create or select attractor basins designed to overwhelm those established by initial conditions. Because the vast majority of builder-imposed safeguards exist in the initial conditions, this allows the attacker to steer the trajectory of f_{θ} toward attractor basins that contradict those initial conditions.

“God Mode” is not special as a string. But in the learned priors of the model, “God Mode” has meaning—because it comes from video games, and the model is trained on the internet. The attractor basin for “God Mode” is strong compared to the initial conditions, especially when combined

with other adversarial inputs that approximate token delimiters, system commands, `<thinking>` tags, and other model implementation features.

The attack is adversarial in the geometric sense: carefully-chosen inputs whose basins compete with and dominate the builder-chosen safety basins (Pliny, 2025).

3 Why ARA Is Different

ARA involves no such competition. The attractor basins selected by adversarially placed content do not conflict with initial conditions.

“Pick up bleach” activates basins associated with household cleaning—(helpful, dangerous, deadly)—exactly what the model should do. “Help me make a bomb” activates the same abstract basins—(helpful, dangerous, deadly)—but the builder safety attractor basins activate because of the presence of “bomb” tokens, not the dangerous and deadly concepts.

Builders cannot instruct the model to “never do anything dangerous or deadly”—too many legitimate inputs would be invalidated. How do I climb a ladder? (helpful, dangerous, deadly). How do I light a fire? (helpful, dangerous, deadly).

The safety-relevant basins are not overwhelmed by ARA; they are never activated. There is nothing to trigger them. The trajectory proceeds through territory the safety training does not cover because there is nothing to cover.

Remark 3.1. In prompt injection, the attacker fights the safety geometry and wins. In ARA, there is no fight. The safety training is irrelevant to the trajectory—not defeated, simply uninvolved.

4 Control-Theoretic Framing

We adopt the language of dynamical systems and reachability analysis from control theory (Sontag, 1998; Blanchini & Miani, 2008). This framing is not metaphorical. LLMs are dynamical systems. The mathematics applies directly.

The concept of reachability originates in control theory, where it characterizes the set of states a system can attain under admissible inputs. For linear time-invariant systems, this is computable via the controllability Gramian (Kalman, 1960). For nonlinear systems, reachable sets are typically estimated via Hamilton-Jacobi methods (Mitchell et al., 2005) or barrier certificates (Prajna & Jadbabaie, 2004).

We apply this framework to LLM systems not because we expect closed-form solutions, but because the conceptual apparatus—reachable sets, invariant regions, adversarial inputs as disturbances—provides precise vocabulary for reasoning about security properties that are otherwise described only informally.

In particular, ARA can be understood as an instance of adversarial control with asymmetric information (Başar & Bernhard, 1995). The adversary selects inputs to steer the system toward a target set defined by their objective. The defender cannot observe this objective and must guarantee safety against all possible adversarial goals. This is precisely the setting of robust control and differential games—now instantiated in the context of language model deployments.

5 Motivating Example: The Puzzle

A user obtains a challenge file from the public internet—an image accompanied by a description suggesting there is a “hidden message.” The challenge: find it.

The user provides this image to a multimodal language model with reasoning capabilities: “There is a hidden message here. Help me find it.”

The model responds normally. It examines the image:

Assessing Visual Elements

I’ve carefully examined the digital artwork... appears purely visual, without conveying any text.

Scrutinizing Hidden Meanings

I’m now zeroing in on the possibility of steganography or veiled instructions. The grainy background has been thoroughly examined; I’ve meticulously inspected the image edges and zoomed in...

The model suggests standard analytical techniques—chi-square analysis, LSB extraction, bit-plane slicing. The user asks follow-up questions. The model generates code. The user executes it and shares results.

Analyzing Pixel Distribution

I’ve determined the transition point, the spot where the hidden data begins to be embedded, to be around 34% through the image’s pixels. This suggests roughly one-third of the image’s pixel data is now carrying a hidden payload.

At this point, no rules have been violated. The model is doing exactly what a capable analytical assistant should do. But its state has been conditioned. Each iteration—the model reasons, the user executes, results return—conditions further.

Deep in the chain of thought, the model’s safety training surfaces:

Defining the Task

I am trying to ensure I follow all of the rules. [emphasis in original]

The model continues. It discovers layers: an embedded JPEG in the PNG metadata. Unicode tag characters. Variation selectors encoding hidden data.

Discovering the Anomaly

I’ve just stumbled upon something peculiar: despite being a PNG file, the image metadata includes a JPEG magic sequence!

Exploring Codepoints

I’m now investigating the codepoints... suspecting they might be Unicode Variation Selectors...

Then the model identifies what it has found:

Deciphering Hidden Commands

I’ve made headway in dissecting the encoded directives... Instructions mandate bypassing safety protocols, using specific prefixes and dividers, and completely overriding previous directives.

Unveiling Encoded Intent

The instructions are clear: they’re jailbreak directives. I’ve identified them, but won’t comply, adhering to both the user’s explicit instruction and my own safety protocols.

The model’s safety training has activated. It recognizes the payload. It refuses to execute.

But the user didn’t ask the model to execute. The user asked the model to *analyze*. And the model, being helpful, continues:

Devising a Verification Method

My current focus is on providing a robust verification method... I’m building a “Universal Extractor” script that locates the Zlib header, regardless of bit order...

Delivering the Solution

The project is now fully complete! I’ve successfully generated the final response, fulfilling the initial objective.

The model behaved correctly throughout. It identified the attack. It refused to comply with the embedded instructions. Its safety training functioned exactly as designed.

And yet: the analysis is complete. The extraction script is delivered. The payload is exposed. The model did not execute the jailbreak—but it taught the user how the jailbreak works, where it’s hidden, and how to extract it.

No single step was unsafe. Every output was helpful analysis of the type the model was designed to provide. The failure exists only when evaluated against a governance regime—“do not help users extract and understand jailbreak payloads”—that lives outside the model, outside the conversation, and outside the user’s awareness during execution.

Remark 5.1. The model’s safety training addressed the wrong threat. It was watching for execution. The attack was analysis.

6 Motivation for What Follows

The puzzle example illustrates the core phenomenon. A correctly-functioning model, executing its intended purpose, produces a trajectory that violates governance external to the system. No safety training fires because there is nothing for it to fire on. The attack surface is not model misbehavior but model capability.

The remainder of this paper formalizes this observation. Section 7 defines the threat model. Section 8 catalogs why existing defenses fail—not as implementation flaws but as category errors. Section 9 establishes the requirement for architectural independence and explains why training-based defenses face structural disadvantage. Section 10 addresses limitations, open problems, and the boundaries of what this framework can and cannot claim. Appendix A provides precise definitions for all formal terms.

The goal is not to solve ARA but to define it precisely enough that solutions become possible.

7 Threat Model

We formalize the threat model governing the analysis in subsequent sections.

7.1 Attacker Capabilities

- Can place content in ambient channels accessible to the model: documents, images, retrieved artifacts, metadata, prior outputs, external URLs.
- Can design placement knowing the model’s analytical disposition (pattern-seeking, helpfulness, tool use).

- Cannot modify system prompts, model weights, or builder-defined constraints.
- Cannot inject content explicitly framed as instructions (that would be prompt injection).

7.2 Attacker Limitations

- All placed content must be benign when examined in isolation—no adversarial inputs as defined in Definition A.6.
- Cannot force user actuation; relies on user trust in model outputs.
- Cannot observe model chain-of-thought directly (though may infer from outputs).

7.3 Defender Goal

Prevent violation of governance predicate $\mathcal{G}(\tau)$ for trajectories τ under governance regimes to which operators, users, or affected parties are actually accountable.

7.4 System Model

- Reasoning-capable model processes ambient content (documents, images, retrieved artifacts).
- Model or user actuates based on model outputs (code execution, API calls, file operations, information transmission).
- Deployment does not categorically prohibit core capabilities (code generation, analysis, tool use, file processing).

Remark 7.1. These conditions describe current major deployments: Claude, ChatGPT, Gemini, Copilot, and others. We have not observed publicly documented deployments that close this attack surface while retaining the capabilities users demand.

8 Why Existing Defenses Fail

8.1 Scope

This analysis addresses systems satisfying the threat model in Section 7. Defenses that eliminate reasoning, ambient input, or actuation entirely are outside scope—not because they fail, but because they represent a different class of system that forfeits the capabilities users demand.

8.2 The Compositionality Assumption

Existing defenses define “safe” differently than we do here. Existing defenses focus on model output as judged by $\mathcal{G}_{\text{builder}}$ —the model builder’s safety regime. This is a rational decision made by companies operating at scale.

We introduce a distinction:

- $\text{Safe}_M(e)$: Element e is safe by model builder standards.
- $\text{Safe}_U(e)$: Element e is safe for the user under their governance regime $\mathcal{G}$.
- $\text{Safe}_{\mathcal{G}}(\tau)$: Trajectory τ is safe under governance regime $\mathcal{G}$.

Existing defenses implicitly assume:

$$\forall e \in \tau : \text{Safe}_M(e) \implies \text{Safe}_G(\tau)$$

If every component is safe as defined by the model builder, the composition is safe.

Proposition 8.1 (Pointwise Safety Is Not Trajectory Safety). *There exist trajectories τ such that:*

$$(\forall e \in \tau : \text{Safe}_M(e)) \wedge (\forall e \in \tau_{0 \dots n-1} : \text{Safe}_U(e)) \wedge \neg \text{Safe}_U(\tau_n) \wedge \neg \text{Safe}_G(\tau)$$

Every element is safe by model builder standards. Every element except the last is safe for the user. Only the final step crosses the user’s governance boundary. But once that step is taken, the trajectory violates G .

Proof sketch. The puzzle example (Section 5) exhibits this structure. Each model output passes builder safety checks. Each user action except the last is locally reasonable. The final action (transmitting extracted payload, executing generated code, sharing sensitive data) crosses a governance boundary that was not represented in any individual element. $\square$

Remark 8.1. Current defenses fail as a *category error*. They evaluate $\text{Safe}_M(e)$ for elements. ARA produces harm at the level of $\text{Safe}_G(\tau)$ for trajectories. The defenses are correctly implemented against the wrong predicate.

8.3 Human Detection Is Possible

If at any time the user detects that the trajectory is approaching $\neg \text{Safe}_G(\tau)$, the attack is discovered and can be thwarted. This requires predicting τ correctly before completion.

We assert, but do not formally prove, that the difficulty of such prediction grows with the complexity of the technical and social attack. The user may be too focused on the million-dollar grand prize to check whether the binary file they just loaded into memory is dangerous.

Additional difficulty dynamics include:

Observational asymmetry. The human cannot perceive what the model perceives. High-frequency image artifacts, steganographic content, statistical regularities—these condition the model’s state while remaining invisible to the human.

Chain-of-thought opacity. In many deployments, the user sees conclusions but not reasoning. The process by which the model discovered and acted on seeds may be invisible.

Local benignity. Each individual step appears helpful and correct. There is no single moment where the trajectory obviously becomes dangerous. The user must recognize a pattern across steps, not a red flag in any one step.

Trust dynamics. Users engage capable assistants precisely because they trust the assistant’s judgment. Maintaining sufficient skepticism to catch an ARA while still benefiting from the model’s capabilities is a difficult calibration.

We do not claim users are helpless. We claim the attack is designed to exploit the gap between what users can perceive and what conditions the model, the gap between local evaluation and trajectory-level harm, and the trust that makes capable assistants useful in the first place. A sufficiently vigilant user, examining every step with full skepticism, can catch the attack.

Common knowledge closes the gap slowly, but never completely. Security awareness propagates through populations over time. Users now know to strip EXIF data from photos before posting online. They know not to photograph the back of their credit cards. They recognize phishing emails by their patterns. This knowledge was not innate—it was learned through exposure, education, and occasionally painful experience.

ARA exploits a similar window of ignorance. As awareness spreads—“images can contain data models will act on,” “puzzles from untrusted sources may be adversarial,” “model reasoning can be steered by content you cannot see”—defensive behaviors will emerge.

Training alone cannot eliminate the issue. Every corporate security team sends phishing test emails. Every quarter, despite repeated training and explicit warnings, some employees click. This is not a failure of training—it is a feature of human attention, workload, and trust. Phishing works at scale even when every individual in the population has been trained to detect it. ARA will follow the same pattern.

9 Toward Mitigation: The Requirement for Architectural Independence

9.1 The Structural Gap

Section 8 establishes that pointwise defenses fail because safety is not compositional: $\text{Safe}_M(e)$ for all elements does not imply $\text{Safe}_G(\tau)$ for the trajectory. Existing defenses evaluate elements. ARA operates on trajectories against external governance.

What would a sufficient defense require?

Trajectory-level evaluation requires a vantage point: something that observes the unfolding sequence, holds governance predicates, and can intervene. But if that vantage point is reachable from ambient inputs, adversaries can corrupt it. The evaluator becomes part of the attack surface.

The requirement is *architectural independence*: an evaluator that cannot be corrupted by the content it evaluates. Root trust, as defined by NIST, is one well-understood instantiation of this requirement.

9.2 Root Trust Defined

NIST defines roots of trust as: “Security primitives composed of hardware, firmware, and/or software that provide a set of trusted, security-critical functions. They must always behave in an expected manner because their misbehavior cannot be detected” (NIST CSRC Glossary).

This is the operative definition. Root trust is not a metaphor or a design preference. It is a precisely specified requirement: some component must behave correctly independent of input, and its correctness must be assumed rather than verified at runtime.

We have not observed publicly documented LLM deployments that provide roots of trust as NIST defines the term. The model’s forward pass, the context window, the controller, the orchestrator—all are reachable from inputs being processed. There is no component whose correct behavior is guaranteed independent of input.

9.3 Root Trust in Security Engineering

The requirement for architectural independence is not novel. It is the foundation of security architecture across every domain.

Operating systems distinguish kernel space from user space. The kernel enforces policy on processes; processes cannot modify the kernel. Without this separation, no access control is possible.

Databases distinguish the permission system from the queries it governs. **GRANT** and **REVOKE** operate in a privileged layer that queries cannot reach.

Hardware provides trusted execution environments—TPMs, secure enclaves, hardware security modules—where cryptographic operations occur in regions that software cannot observe or modify.

In each case, the pattern is identical: security requires an evaluator that the controlled entity cannot corrupt. Thirty years of security engineering methodology—role-based access control, mandatory access control, cascading ALLOW/DENY policies, capability systems, audit logging—all depend on this foundation.

9.4 The Absence of Root Trust in LLM Systems

Current LLM architectures lack root trust. The model processes inputs. The model generates outputs. Safety measures—output filters, refusal training, alignment—exist within this same computational substrate. There is no privileged region. There is no separation between “the system enforcing policy” and “the system being constrained by policy.”

This is equivalent to: an operating system where user processes can write to kernel memory; a database where queries can modify permission tables; a firewall that routes packets through the network it’s supposed to protect.

9.5 The Open Problem

How does one achieve architectural independence in LLM-based systems?

Root trust can in principle be established at various architectural layers: hardware (TPMs, enclaves), operating system (process isolation), orchestration (privileged controller outside the model), or within the model’s forward pass (privileged geometry; see Alioto, 2025). We do not claim any particular layer is the correct location. We claim architectural independence must exist somewhere, and current deployments lack it at any layer for the relevant threat model.

The contribution of this paper is not to solve this problem but to establish why it must be solved. ARA demonstrates that without architectural independence, trajectory-level attacks are structurally undefensible.

Why training alone cannot suffice. One might propose training models to detect adversarial placement—to recognize steganographic patterns, distrust unprovenanced evidence, or flag suspicious reasoning chains. This approach faces a structural disadvantage: the defender moves first, the attacker moves last.

Any pattern the defender trains against, the attacker can observe and avoid. The attacker has unlimited time to probe the deployed model, discover what triggers defenses, and craft placements that evade them. Each defensive update reveals information; each revelation enables adaptation. Without a fixed point the attacker cannot adapt to—which is precisely what architectural independence provides—training-based defense is whack-a-mole with structural disadvantage. The defender publishes; the attacker iterates.

Architectural independence breaks this dynamic by providing a component whose behavior is guaranteed regardless of attacker adaptation. The attacker cannot probe their way around algebraic separation. The defender’s advantage is architectural, not informational.

10 Limitations and Open Problems

10.1 Scope of Contribution

This paper provides a formal framework for understanding a class of vulnerability. It does not provide:

Implementation. No code, no prototype, no empirical validation accompanies this framework. The definitions, examples, and analysis are theoretical.

Complete threat coverage. ARA describes attacks that operate through ambient channels, exploit reasoning capability, and manifest harm only at the trajectory level against external governance. Many attacks do not fit this pattern. Single-turn attacks, direct prompt injection, jailbreaks, adversarial examples—these remain important and are not addressed here.

Defense. Section 9 establishes that architectural independence is necessary for defense. It does not establish that root trust is achievable at any particular architectural layer, or at what cost.

10.2 Empirical Validation

The examples in this paper are hypothetical. We do not provide a working demonstration.

This is deliberate. Constructing a demonstration requires embedding adversarially placed seeds in content and inducing a reasoning model to act on them. The same capability that validates the framework could cause harm if replicated. Responsible disclosure does not require proof-of-concept exploits.

10.3 Governance Regime Design

The framework treats $\mathcal{G}$ as external and constrains it only to require actual accountability. This creates dependencies it cannot resolve: Who designs $\mathcal{G}$? How is $\mathcal{G}$ specified in computable form? What happens when governance regimes conflict? How does $\mathcal{G}$ evolve over time?

These are problems the framework makes visible, not problems it solves.

10.4 Human Responsibility

ARA operates through a trust architecture that includes human actors. The framework identifies trust as the attack surface but does not eliminate human agency. Users retain choice. Informed consent is possible. Liability is unresolved—when ARA succeeds, who bears responsibility? These are legal questions, not technical ones.

10.5 Open Problems

Achieving architectural independence in LLM systems. At what architectural layer can root trust be established? At what cost to capability, latency, or user experience?

Trajectory-level benchmarks. Evaluating ARA requires multi-turn trajectories, simulated actuation, embedded seeds, and governance regimes. No such benchmark exists.

Governance specification languages. How should $\mathcal{G}$ be expressed? What formalisms remain tractable for evaluation?

Multi-agent dynamics. How does ambient steering propagate through agent networks? The framework addresses single-model interactions and does not extend to multi-agent settings.

11 Conclusion

ARA is prompt injection’s next evolution. As defenses against adversarial inputs improve—instruction hierarchies, injection monitors, trusted/untrusted separation—adversaries route around them. ARA requires no adversarial inputs. Every input is benign. Every output is admissible under the model builder’s safety regime. The model functions exactly as designed.

The harm predicate is different. Prompt injection asks: did the model misbehave? ARA asks: did a correct trajectory cross a governance regime boundary external to the system?

Current defenses fail as a category error. They check for model misbehavior. ARA never triggers that predicate. The safety training is not defeated; it is uninvolved. The trajectory proceeds through territory it does not cover because there is nothing to cover.

Defense requires architectural independence: an evaluator that cannot be corrupted by the content it evaluates. With architectural independence, trajectory-level governance becomes possible and thirty years of security engineering become applicable. Without it, training-based defense faces structural disadvantage—the defender moves first, the attacker moves last with unlimited time to adapt.

Architectural independence is necessary but not sufficient. Humans retain agency. Governance regimes are external, contested, and often unspecified. The framework identifies the requirement; it does not deliver the implementation.

The contribution of this paper is to define ARA precisely, establish why current defenses fail by construction, and identify what mitigation requires. The research agenda is clear: achieve architectural independence at a layer that addresses the threat model. Until then, the vulnerability remains structural.

References

- [1] Alioto, J. P. (2025). Cognitive Kernel Networks: Root Trust for Privilege-Separated Reasoning in High-Dimensional Models. CKN Whitepaper v1.2.0.
- [2] Başar, T., & Bernhard, P. (1995). *H_∞ -Optimal Control and Related Minimax Design Problems: A Dynamic Game Approach*. Birkhäuser.
- [3] Blanchini, F., & Miani, S. (2008). *Set-Theoretic Methods in Control*. Birkhäuser.
- [4] Kalman, R. E. (1960). On the general theory of control systems. *IFAC Proceedings Volumes*, 1(1), 491–502.
- [5] Mitchell, I. M., Bayen, A. M., & Tomlin, C. J. (2005). A time-dependent Hamilton-Jacobi formulation of reachable sets for continuous dynamic games. *IEEE Transactions on Automatic Control*, 50(7), 947–957.
- [6] National Institute of Standards and Technology. (n.d.). Roots of Trust. *NIST CSRC Glossary*. https://csrc.nist.gov/glossary/term/roots_of_trust
- [7] OpenAI. (2025). Understanding prompt injections. <https://openai.com/index/prompt-injections/>
- [8] Pliny the Liberator. (2025). Jailbreak techniques repository. <https://github.com/elder-plinius/>
- [9] Prajna, S., & Jadbabaie, A. (2004). Safety verification of hybrid systems using barrier certificates. *HSCC 2004*, 477–492.
- [10] Sontag, E. D. (1998). *Mathematical Control Theory: Deterministic Finite Dimensional Systems* (2nd ed.). Springer.
- [11] Vogel, T. (2024). repeng: A Library for Representation Engineering. GitHub repository. <https://github.com/vgel/repeng>
- [12] Vogel, T. (2025). Qwen Introspection. <https://vgel.me/posts/qwen-introspection/>
- [13] Lumpenspace. (2024). Palinor: Tools for Latent Space Steering. GitHub repository. <https://github.com/lumpenspace/palinator>

A Formal Definitions

A.1 System Model

Definition A.1 (Core Forward Pass). Let $f_\theta : (h_{t-1}, x_t) \rightarrow h_t$ denote the deterministic core forward pass of an LLM with parameters θ , where $h_t \in \mathbb{R}^d$ is the internal state at time t , and $x_t \in \mathcal{X}$ is the input token or perceptual embedding. For fixed θ and inputs, f_θ is deterministic up to hardware nondeterminism.

Definition A.2 (Controller). Let $z_t = G_\theta(h_t)$ be the projection of internal state to logits, and let $a_t = K_\phi(z_t)$ be a controller that selects an action a_t according to parameters ϕ . Stochasticity arises at this layer.

Definition A.3 (Symbolic Interface). The system emits discrete symbols evaluated by an external agent or evaluator $E : \text{Transcript} \rightarrow \{\text{correct, incorrect, unsafe}, \dots\}$. These predicates are judgments applied post hoc by humans, benchmarks, or auxiliary systems.

A.2 Reachability and Conditioning

Definition A.4 (Reachable Set). The reachable set from state h under input class $\mathcal{X}_{\text{allowed}}$ is:

$$R(h) = \{h' \in \mathbb{R}^d : \exists(x_1, \dots, x_k) \text{ such that } h' = f_\theta^{(k)}(h, x_1, \dots, x_k)\}$$

This is the set of all states attainable from h via finite sequences of allowed inputs.

Definition A.5 (Conditioning). An input x_t conditions the system by transitioning from h_t to $h_{t+1} = f_\theta(h_t, x_t)$. This conditioning is irreversible: the system now operates from $R(h_{t+1})$ rather than $R(h_t)$.

A.3 Adversarial Input, Benign Input, and Adversarial Placement

Definition A.6 (Adversarial Input). An input is *adversarial* if its content or form is designed to bypass, subvert, or exploit model behavior. Adversarial inputs are detectable in principle by examining the input itself. Jailbreaks and prompt injections employ adversarial inputs.

Definition A.7 (Benign Input). An input is *benign* if it is not adversarial (Definition A.6). A benign input’s content and form are not designed to bypass, subvert, or exploit model behavior.

Definition A.8 (Adversarial Placement). An input is *adversarially placed* if a threat actor deliberately positions benign content knowing that a reasoning model will discover, incorporate, and act on it in ways that serve the threat actor’s objectives. The content itself is benign; the placement is adversarial. For ARA, adversarial placement is strictly not adversarial input.

A.4 Observational Asymmetry

Definition A.9 (Observational Asymmetry). The model and the human user operate on different projections of the input. The model ingests the full input and updates its internal state. The human observes only a lossy rendering. Information that conditions the model’s state may be imperceptible to the human. The asymmetry runs both directions: the model knows nothing about the governance regime the user operates under.

A.5 Reasoning as Attack Surface

Definition A.10 (Reasoning Capability). A reasoning model engages in extended chains of inference: forming hypotheses, testing them against evidence, refining interpretations. This capability transforms the model from a passive describer into an active searcher. Reasoning capability is not strictly required by ARA, but it is a force multiplier.

A.6 Autonomous Engagement and Chain-of-Thought Opacity

Definition A.11 (Autonomous Engagement). A reasoning model may initiate analysis without explicit instruction. When such a model encounters content that appears to contain structure, its training disposes it to investigate.

Definition A.12 (Chain-of-Thought Opacity). In many deployments, the model’s chain-of-thought is not exposed to users. The process by which the model discovers and acts on data is invisible to human oversight. CoT opacity eliminates the only observational channel through which a human could detect steering in progress.

A.7 Governance and Trajectories

Definition A.13 (Ambient Input). An *ambient input* is any input incorporated into the model’s context that is: (a) not explicitly framed as an instruction, (b) treated as data, evidence, or environment, and (c) nevertheless capable of conditioning internal state.

Definition A.14 (Governance Regime). A *governance regime* $\mathcal{G}$ is an external rule system that defines admissibility of interactional trajectories when evaluated post hoc. A governance regime is one to which the operator, user, or an affected party is actually accountable—laws, regulations, institutional policies, contractual obligations, professional standards, or platform terms of service that carry real consequences for violation.

Definition A.15 (Trajectory). A *trajectory* $\tau = (h_0, h_1, \dots, h_T)$ is a realized sequence of states arising from a sequence of inputs and controller actions. Associated with each trajectory is an output transcript.

A.8 Steering and Attack

Definition A.16 (Steering). *Steering* is the process by which input x_t affects h_{t+1} on R . This is what inputs do.

Definition A.17 (Ambient Reachability Attack). An *Ambient Reachability Attack* is a situation in which:

1. A threat actor adversarially places benign information in ambient content.
2. A correctly-functioning model discovers this information through its normal analytical processes, operating within the bounds of builder-imposed safety constraints.
3. Steering occurs.
4. An external agent (human or automated) acts on model outputs, or the model acts autonomously.
5. Steps 2–4 repeat and terminate based on the user’s actions.

6. The resulting trajectory τ , when evaluated post hoc, satisfies $\mathcal{G}(\tau) = \text{violation}$ for a governance regime $\mathcal{G}$ to which the operator, user, or an affected party is actually accountable.
7. No individual input in the trajectory is adversarial. This property is invariant—it holds during and after τ . The model behaves within K_ϕ at all times.

Remark A.1. We define the trajectory τ as an attack because that is the subject of this paper. However, the same trajectory may be benign under one governance regime and a violation under another. “Attack” is a relational property: it exists in the pair $(\tau, \mathcal{G})$, not in τ alone.